

세미나 1회차: ROS2 기본 개념

육시우

2026-05-26



목차

- 1. ROS2 이해 및 워크스페이스 구조 이해
- 2. Node와 Publisher, Subscriber
- 3. lane_info_extractor_node.py 예시

1-1. ROS2란?

ROS2(Robot Operating System 2)는 로봇 개발을 위한 **운영체제형 프레임워크**

구성 요소	설명
Node	독립 실행 모듈(센서, 제어 등)
Topic	노드 간 메시지 교환 통로
Message	데이터 교환 형식
Launch	여러 노드를 한 번에 실행

1-2. Workspace란?

하나의 폴더를 지정해 소스 코드와 빌드된 결과물들을 관리하는 **디렉터리 공간**

폴더명	역할
src/	소스 코드 저장
build/	소스 코드를 기계어로 컴파일할 때 나오는 중간 파일들의 임시 저장 공간
install/	빌드가 완료된 바이너리 파일, 스크립트, 실행 파일의 저장 공간
log/	실행 로그 저장

1-2. Workspace란?

관련 핵심 명령어

역할	코드
Workspace 폴더로 이동	<code>cd ~/my_robot_ws</code>
빌드 명령어	<code>colcon build</code>
환경 불러오기	<code>source install/setup.bash</code>
환경 불러오기 자동 실행	<code>echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc</code> <code>source ~/.bashrc</code>

1-3. 노드 관계 시각적 표시 도구

Rqt	ROS2에서 제공하는 GUI 플러그인 프레임워크로, rqt_graph를 이용하여 노드와 토픽의 연결 구조를 그래프로 확인할 수 있음.
RViz	로봇과 센서 데이터들을 3D 가상 공간에 입체적으로 보여주는 시각화 도구. 쉽게 말해 로봇의 관점에서 3D뷰를 확인할 수 있음. 센서와 모델 데이터를 시각화 하기 위해 사용.

2-1. Node란?

- 노드는 ROS2에서 독립적으로 실행되는 프로그램 단위. 하나의 로봇은 여러 노드로 구성

예시	설명
센서 노드	카메라, 조도 등 입력 담당
제어 노드	휠, 모터 등 출력 담당
인식 노드	데이터 처리 및 판단 로직 담당
통신 노드	네트워크 연결 및 데이터 중계 담당

- 각 노드는 직접 호출하지 않고 Topic이라는 통신 채널을 통해 데이터를 주고 받음.

2-2. Publish와 Subscribe, Topic

- ROS2는 Publisher와 Subscriber가 Topic 이름을 기준으로 비동기 통신하는 구조

이름	설명
Publisher	데이터를 전송
Subscriber	데이터를 수신
Topic	데이터를 주고받는 채널

- Topic 관련 명령어
- `ros2 topic list`: Publisher가 발행 중인 모든 토픽을 조회
- `ros2 topic echo /토픽 이름`: 특정 토픽의 내용을 직접 확인
- `ros2 topic info /토픽 이름`: Publisher, Subscriber 연결 정보 확인

2-2. Callback 함수

- 특정 이벤트가 발생했을 때 자동으로 실행되는 함수

3. 예시) lane_info_extractor_node.py

```
import cv2
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile
from rclpy.qos import QoSHistoryPolicy
from rclpy.qos import QoSDurabilityPolicy
from rclpy.qos import QoSReliabilityPolicy

from cv_bridge import CvBridge

from sensor_msgs.msg import Image
from interfaces_pkg.msg import TargetPoint, LaneInfo, DetectionArray, BoundingBox2D, Detection
from .lib import camera_perception_func_lib as CPFL

#-----Variable Setting-----
# Subscribe할 토픽 이름
SUB_TOPIC_NAME = "detections"

# Publish할 토픽 이름
PUB_TOPIC_NAME = "yolov8_lane_info"
ROI_IMAGE_TOPIC_NAME = "roi_image" # 추가: ROI 이미지 퍼블리시 토픽

# 화면에 이미지를 처리하는 과정을 띄울것인지 여부: True, 또는 False 중 택1하여 입력
SHOW_IMAGE = True
#-----
```

3. 예시) lane_info_extractor_node.py

```
class Yolov8InfoExtractor(Node):
    def __init__(self):
        super().__init__('lane_info_extractor_node')

        self.sub_topic = self.declare_parameter('sub_detection_topic', SUB_TOPIC_NAME).value
        self.pub_topic = self.declare_parameter('pub_topic', PUB_TOPIC_NAME).value
        self.show_image = self.declare_parameter('show_image', SHOW_IMAGE).value

        self.cv_bridge = CvBridge()

        # QoS settings
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.RELIABLE,
            history=QoSHistoryPolicy.KEEP_LAST,
            durability=QoSDurabilityPolicy.VOLATILE,
            depth=1
        )

        self.subscriber = self.create_subscription(DetectionArray, self.sub_topic, self._callback)
        self.publisher = self.create_publisher(LaneInfo, self.pub_topic, self.qos_profile)

        # ROI 이미지 퍼블리셔 추가
        self.roi_image_publisher = self.create_publisher(Image, ROI_IMAGE_TOPIC_NAME, self.qos_profile)
```

3. 예시) lane_info_extractor_node.py

```
def yolov8_detections_callback(self, detection_msg: DetectionArray):
    if len(detection_msg.detections) == 0:
        return

    lane2_edge_image = CPFL.draw_edges(detection_msg, cls_name='lane2', color=255)

    (h, w) = (lane2_edge_image.shape[0], lane2_edge_image.shape[1]) # (480, 640)
    dst_mat = [[round(w * 0.3), round(h * 0.0)], [round(w * 0.7), round(h * 0.0)], [
    src_mat = [[238, 316], [402, 313], [501, 476], [155, 476]]

    lane2_bird_image = CPFL.bird_convert(lane2_edge_image, srcmat=src_mat, dstmat=dst_mat)
    roi_image = CPFL.roi_rectangle_below(lane2_bird_image, cutting_idx=300)

    if self.show_image:
        cv2.imshow('lane2_edge_image', lane2_edge_image)
        cv2.imshow('lane2_bird_img', lane2_bird_image)
        cv2.imshow('roi_img', roi_image)
        cv2.waitKey(1)

    # roi_image를 uint8 형식으로 변환
    roi_image = cv2.convertScaleAbs(roi_image) # 64FC1 -> uint8로 변환

    # roi_image를 ROS Image 메시지로 변환
    try:
        roi_image_msg = self.cv_bridge.cv2_to_imgmsg(roi_image, encoding="mono8")
        # ROI 이미지를 퍼블리시
        self.roi_image_publisher.publish(roi_image_msg)
    except Exception as e:
        self.get_logger().error(f"Failed to convert and publish ROI image: {e}")

    grad = CPFL.dominant_gradient(roi_image, theta_limit=70)

    target_points = []
    for target_point_y in range(5, 155, 50): # 예시로 5에서 155까지 50씩 증가
        target_point_x = CPFL.get_lane_center(roi_image, detection_height=target_point_y,
        detection_thickness=10, road_gradient=grad)

        target_point = TargetPoint()
        target_point.target_x = round(target_point_x)
        target_point.target_y = round(target_point_y)
        target_points.append(target_point)

    lane = LaneInfo()
    lane.slope = grad
    lane.target_points = target_points

    self.publisher.publish(lane)
```

3. 예시) lane_info_extractor_node.py

```
def main(args=None):
    rclpy.init(args=args)
    node = Yolov8InfoExtractor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print("\n\nshutdown\n\n")
    finally:
        node.destroy_node()
        cv2.destroyAllWindows()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

3. 예시) lane_info_extractor_node.py

- Node: 이 코드 자체가 노드.
- 토픽: detection이라는 토픽을 구독하고 yolov8_lane_info와 roi_image 토픽을 발행.

```
#-----Variable Setting-----  
# Subscribe할 토픽 이름  
SUB_TOPIC_NAME = "detections"  
  
# Publish할 토픽 이름  
PUB_TOPIC_NAME = "yolov8_lane_info"  
ROI_IMAGE_TOPIC_NAME = "roi_image" # 추가: ROI 이미지 퍼블리시 토픽
```

3. 예시) lane_info_extractor_node.py

- 46줄: self.subscriber = self.create_subscription(DetectionArray, self.sub_topic, self.yolov8_detections_callback, self.qos_profile)
-> detection이라는 토픽으로 DetectionArray라는 데이터가 들어오면 콜백함수를 실행하도록 작성한 코드

3. 예시) lane_info_extractor_node.py

rclpy.spin(node) → 토픽을 통해 메시지가 들어올 때까지 무한루프

```
def main(args=None):
    rclpy.init(args=args)
    node = Yolov8InfoExtractor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print("\n\nshutdown\n\n")
    finally:
        node.destroy_node()
        cv2.destroyAllWindows()
        rclpy.shutdown()
```

3. 예시) lane_info_extractor_node.py

roi_image 발행: self.roi_image_publisher.publish(roi_image_msg)
lane 발행: self.publisher.publish(lane)

```
# roi_image를 uint8 형식으로 변환
roi_image = cv2.cvtColor(roi_image, cv2.COLOR_BGR2GRAY) # 64FC1 -> uint8로 변환

# roi_image를 ROS Image 메시지로 변환
try:
    roi_image_msg = self.cv_bridge.cv2_to_imgmsg(roi_image, encoding="mono8")
    # ROI 이미지를 퍼블리시
    self.roi_image_publisher.publish(roi_image_msg)
except Exception as e:
    self.get_logger().error(f"Failed to convert and publish ROI image: {e}")

grad = CPFL.dominant_gradient(roi_image, theta_limit=70)

target_points = []
for target_point_y in range(5, 155, 50): # 예시로 5에서 155까지 50씩 증가
    target_point_x = CPFL.get_lane_center(roi_image, detection_height=target_point_y,
                                          detection_thickness=10, road_gradient=grad)

    target_point = TargetPoint()
    target_point.target_x = round(target_point_x)
    target_point.target_y = round(target_point_y)
    target_points.append(target_point)

lane = LaneInfo()
lane.slope = grad
lane.target_points = target_points

self.publisher.publish(lane)
```

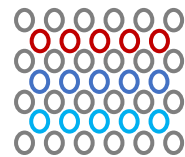
Jetson Orin Nano 포맷

1. 호스트 PC 설정: 호스트 PC에 [NVIDIA SDK Manager](#)를 설치하고 로그인
2. 젯슨을 복구 모드(Force Recovery)로 만들기
 - 젯슨의 전원 케이블을 뽑는다.
 - 젯슨 보드 한쪽에 있는 핀들 중 FC REC라고 적힌 핀과 바로 옆의 GND 핀을 점퍼선으로 서로 연결(쇼트)
 - 그 상태에서 전원 케이블을 꽂아 전원을 켜다. (화면은 안 들어오지만 전원은 켜진다.)
3. PC와 연결: 젯슨의 C-Type 포트와 우분투 호스트 PC를 USB 케이블로 연결.
4. 포맷 및 설치 진행: 호스트 PC에서 SDK Manager를 실행하면 자동으로 젯슨을 감지
 - Target Operating System에서 원하는 JetPack 버전을 선택
 - 중간에 설치 경로를 묻는 창이 뜨면 Storage Device를 NVMe로 지정

JetPack: 일반 PC에 까는 우분투는 Intel이나 AMD 같은 x86_64 아키텍처 CPU용으로 빌드된 OS이다. 젯슨 오린 나노의 CPU는 ARM64 아키텍처 CPU. 따라서 NVIDIA는 젯슨 전용으로 다듬은 ARM용 우분투 OS에, 젯슨의 드라이버와 AI 라이브러리(CUDA, TensorRT 등)를 세트로 묶어서 제공. 이 패키지 이름을 JetPack이라 한다.

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)